Ahmedabad University

CSC530 – Introduction to Prompt Engineering

MID-SEMESTER REPORT

Group Number	4
Group Members	Akkshat Shah - AU2340246 Devasya Desai - AU2320161 Kesar Vipul Thakkar - AU2420041
Date of Submission	12-09-2026

1. Introduction

This report outlines the development process of Group 4's project, an Inquiry-Based Commerce Mentor, which is a Retrieval-Augmented Generation (RAG) tool created for an assignment titled "Design and Develop a RAG Based Tool Using the TOCD Framework". It is not just a chatbot but a tool that responds to the questions of NCERT Class 11/12 Commerce (Accountancy and Business Studies) based on only a specific set of projects. It uses only a local Ollama LLM (qwen2.5:3b for generation, nomic-embed-text for embeddings) and ChromaDB (as the vector store), with a FastAPI backend and a lightweight web UI. We implemented 4 different templates – TOCD, RTAO, Ultimate and we also implemented normal wrappers to reduce the ambiguity.

2. Planned Activities

- To Curate a Commerce knowledge base, create a document processing pipeline (clean, chunk, embed, vector store).

  → It was then that we analyzed the NCERT commerce syllabus to understand what students actually ask for and where a bare LLM hallucinates and then wrote original topic notes as opposed to copying books. We tried out cloud APIs vs local models, and migrated to Ollama (qwen2.5:3b + nomic-embed-text), to get a free, offline, reproducible setup, and then created the Chunking pipeline (ChromaDB) that takes a clean text, chunks it up, embeds it, and stores it.

- Create the RAG core: retrieval component, four different prompt templates (TOCD, RTAO, Ultimate, Normal), and five supporting wrappers.

  → Once the vector store was in place, we created a simple cosine-similarity retrieval model, and then started to create the RAG architecture, which had 4 switchable prompt templates so that the prompt-design aspect of the assignment was tested, not just one prompt. Testing revealed certain failure modes: off-topic questions, guessing on blank retrieval, ignoring ambiguity, specialization, and personalization, and each one was wrapped up in its own (domain guard, ambiguity, groundedness, specialization, personalization).

- Building the Web interface and API layer (FastAPI endpoints, streaming response, source/grounding display).

  → In order to be able to query RAG end-to-end from the browser, we added a FastAPI layer to it, which allows it to be called as a script. For CPU inference, an early full-response endpoint felt too slow; thus, we redesigned it into a streaming endpoint, called `/chat/stream`, that has a live source/groundedness display.

- Create and conduct formal assessment (15 formal test items: simple, difficult, no information, hallucination-probe).

  →The evaluation was designed after the system was working because we wanted to intentionally divide the questions into failure modes. This allowed us to test recall/grounding (straightforward, complex) separately from the domain guard/insufficient-info wrapper, as well as test the domain guard/insufficient-info wrapper separately from the hallucination resistance.

3. Methodology

The project's pipeline is as follows: User Query → Web Interface → Query Processing (Mode Switch API, Domain/Ambiguity Wrapper) → Retrieved Context (ChromaDB) → LLM Generation (Ollama) → Groundedness Check API → Response → User. The documents are cleaned, chunked with sentence-aware sliding-window chunking (chunk size ~900 characters, overlap ~150 characters) and inserted into a permanent collection in ChromaDB with cosine similarity. At query time, a lightweight keyword-based domain guard filters out irrelevant questions before it makes any call to an LLM, an ambiguity check prompts clarifying questions on ambiguous queries, and a specialization wrapper determines the retrieval breadth (top-k) based on the breadth or narrowness of the query. Chunks retrieved are fed into one of four prompt templates (specified by the :tocd, :rtao, :ultimate, :normal keywords), passed through LLM and then grounded and returned with their source tags. LRU cache: If the same query comes again, the cache does not re-compute the result. The frontend components of the team were implemented in Python, Fast API, ChromaDB, Ollama, and vanilla JS/HTML/CSS, using Git for version control and manual/scripted testing on the stubbed-LLM harness for verifying the prompt logic, offline.

4. Status of Planned Activities

Activity 1: Knowledge Base Curation & Document Processing Pipeline

Status: Complete

Implementation: "`ingest.py`" cleans, chunks (sentence-aware, ~900 chars/150 overlap), embeds via Ollama, and stores everything in a persisted ChromaDB collection; supports both `.txt` and `.pdf` sources.

Difficulties: Balancing chunk size/overlap — too small broke sentence context, too large hurt retrieval precision

Activity 2: RAG Core Engine (Retrieval + Prompt Templates + Wrappers)

Status: Complete

Implementation: `rag_engine.py` (retrieval, caching, session state), `templates.py` (TOCD/RTAO/Ultimate/Normal), `wrappers.py` (domain guard, ambiguity, groundedness, specialization, personalization).

Difficulties: Found the personalization wrapper's profile note was computed but never reaching the LLM prompt — fixed by wiring it into all four templates.

Activity 3: Web Interface & API Layer

Status: Complete.

Implementation: FastAPI backend (`/chat`, `/chat/stream`, `/health`, `/reindex`) with a streaming NDJSON response and a JS frontend showing sources/groundedness live.

Difficulties: Initial non-streaming endpoint felt slow on CPU inference; solved by switching to token-level streaming.

Activity 4:Formal Evaluation (15 Test Items)

Status: Partially completed

Implementation: "`eval/test_questions.json` "(5 straightforward, 5 complex, 3 insufficient-info, 2 hallucination-probe) and "`eval/run_eval.py`;" a mocked sample report exists to show expected shape

Difficulties: Final knowledge base is still being expanded, and we want the KB stable before generating results worth reporting on.

5. Individual Contributions

Member 1 Akkshat Shah (AU2340246) :

Contribution: Owned Activities 1 and 2: knowledge base pipeline and the RAG core. Created ingest.py (clean → chunk → embed → store), designed rag_engine.py, templates.py, wrappers.py, the session/profile dataclasses, the LRU response cache, all four prompt templates, and all five wrappers (domain guard, ambiguity, groundedness, specialization, personalization). Identified and addressed the issue related to the computation but never inclusion of the personalization wrapper's profile note in the LLM prompt; also restructured the ultimate template to the full 8-part skeleton with the profile as [Role]. Exercised all four modes and wrappers offline with a stub-Ollama harness (fake embeddings + fake chat). This means that retrieval, all four templates, and personalization function correctly end-to-end.

Member 2 Kesar Vipul Thakkar (AU2420041) :

Contribution: Owned Activity 3 — the web interface and API layer. Implements the FastAPI backend: /chat, /chat/stream, /health, /reindex; the Pydantic request/response models; the streaming NDJSON generator; the frontend: app.js that parses the stream and displays source/groundedness/cache tags for the tokens. The initial non-streaming endpoint had some latency issues on CPU inference, solved by redesigning the main chat flow to account for token streaming. Tested manually using curl and browser action, and able to see that the stream is correct in splitting the JSON lines. The app, therefore, will stream the responses live with grounding visible per response.

Member 3 Devasya Desai (AU2320161) :

Contribution Made: Owned Activity 4: evaluation framework and retrieval quality. Built eval/run_eval.py and authored all 15 adversarial test cases in eval/test_questions.json across four categories. Tuned CHUNK_SIZE, CHUNK_OVERLAP, and MIN_RELEVANCE_SCORE in config.py to 900/150/0.35, and implemented cosine distance to similarity conversion with relevance filtering in rag_engine.py. Added knowledge_base/marketing_and_management.txt and fixed sentence-aware chunking in ingest.py to prevent oversized, semantically mixed chunks. Validated retrieval, groundedness, latency, and hallucination resistance end-to-end.

6. General Status of the Project

The prototype is end-to-end including knowledge base, document processing pipeline, retrieval, four selectable prompt templates, five wrappers and streaming FastAPI/JS web UI, running locally against Ollama. Activities 1-3 are “done” and manually checked (knowledge base + pipeline, RAG core, web/API layer) including the fix that allows personalization to be sent to the LLM prompt. We have built the harness and all 15 test questions for Activity 4 to make the formal evaluation run for the live system as we want to complete our knowledge base before any results are generated to report on. The team is on time: There is no blocked or dropped planned activity, and there is no work that has been assigned to be done in the core architecture.

7. Upcoming Activities

- A few few-shot examples for each template. Currently the four templates are all zero-shot (instructions only no worked examples). To reduce the format drift and improve the turn-to-turn consistency of a model's structured output, insert 1-2 examples of Q→A pairs (in TOCD/RTAO/Ultimate format) directly into the system prompt.
- Chain of verification (self-critique) in groundedness. The groundedness wrapper is a heuristics / keywords overlap, but not a real verification step. Replace/augment with a Chain-of-Verification prompt – model drafts an answer, then re-prompts to verify the retrieved context sentence by sentence before finalizing – a more robust, prompt-engineering based hallucination check than string matching.
- Multi-step questions prompting with query rewriting / decomposition Add a query-decomposition prompt breaking a compound query into sub-queries before retrieval to the “complex” category (i.e., multi-step/comparison questions); this should directly benefit retrieval recall for our multi-step test questions.
- Structured (JSON-mode) output prompting. The sources/groundedness are currently inferred by using the wrapper layer out of prompt. How to teach a model to write a tiny json block (answer, sources_used, confidence) along with a prose answer and let the user don't use regex/heuristic parsing? It needs to adopt different kinds of prompt engineering methods.
- Systematic prompt analysis (Temperature and template sweep) If the actual test (15 questions) was conducted previously, let prompt design be another variable, by running the questions again at different temperatures, with various templating and testing groundedness and hallucination, such that it becomes a legitimate probe engineering ablation study, rather than just a fixed probe test.